C++ Should Be C++

Document number: P3023R1
Date: 2023-10-31
Authors: David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>
Audience: Evolution, Library Evolution

Abstract

Over the past few years, the C++ community has coped with challenging social media situations, calls for a so-called successor, and signs of upcoming anti-C++ safety regulations. This piles on top of the ordinary committee stress of competing designs and prioritization difficulties. In a time like this it's easy to dwell on troubles or adopt fiercely defensive positions.

This paper attempts to reframe unconstructive narratives and argue that the committee's real opportunity is to improve people's lives. We'll show how this outlook leads to guidance on committee participation, direction, and responsibilities.

Introduction

The C++ community has been through a lot over the past few years. This paper makes no attempt to recount this history (God forbid!), but instead acknowledges that present circumstances have led committee members to question "Why are we here?", "Is participation in C++ standardization still worthwhile?", "What does the future hold for C++?", and "Where should I invest my efforts?". Today's answers to these questions will define C++ for years to come.

This document makes the case for my personal perspective and the technical directions that flow from it. My opinions are formed from 23 years of industry C++ experience and 8 years of active committee participation. Equally contributing are countless conversations with engineers flowing from participation in the Boost Foundation, the Bloomberg C++ Guild, C++ conferences, and #include<C++>. However, I don't pretend to have all the answers and maintain the right to change my mind when new information presents itself.

This document is split into the three parts. The first considers different ideas for a C++ Standardization Committee mission and argues that the most compelling one is to improve people's lives. The second discusses some social patterns and temptations that lead to

counter-productive decisions. The final part focuses on technical biases and considers some of the immediate choices the committee faces.

Acknowledgements

Some of what I say here has been more eloquently said elsewhere. I refer the reader especially to *Direction for ISO C++* (P2000R4) (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2000r4.pdf) from the direction group and *Thriving in a Crowded and Changing World* (https://dl.acm.org/doi/pdf/10.1145/3386320) from Bjarne Stroustrup. I'll be quoting extensively from these documents. *How can you be so certain?* (P1962R0) (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1962r0.pdf) and *Operating principles for evolving C++* (P0559R0) (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0559r0.pdf) also cover similar topics.

I'd be remiss if I did not also credit Niall Douglas, Inbal Levi, Bjarne Stroustrup, and Herb Sutter for providing valuable feedback that substantially improved this document.

Mission

The biggest threat

How can you be so certain? states "if we are not careful, C++ can still fail". *Operating principles for evolving C++* suggests principles "in order to keep C++ alive, healthy and progressing". These statements, representing a wider community outlook, imply C++ is an entity that has acquired something valuable that can be lost. What does this mean, exactly?

It is easy to see that C++ is fit as a general-purpose programming language–adoption by millions is a testament to that. Engineers gain proficiency in a reasonable amount of time and use it to solve their problems. C++'s usefulness it what matters.

Many think other programming languages "threaten" C++ or have the potential to "take away" what it has. A closer look reveals that the presence of a new tool doesn't make an existing tool less capable, but potentially more capable.

What about regulatory legislation? It cannot change C++'s capabilities any more than those of my hammer. C++ does what it does and is useful where it is used. Unlike my hammer, however, there *is* an entity with the power to degrade C++'s fitness: the C++ standardization committee.

The surest way to make a standard irrelevant is to say yes to everything. It is easy to see why: standardizing a complex mess of incoherent ideas quickly becomes "insanity to the point of endangering the future of C++"[1]. If my hammer is replaced with a complex gadget requiring thousands of pages of instructions, it is no longer useful for me. Yet, this is exactly our tendency with C++.

If C++'s biggest threat is the standardization committee then that begs the question of how to mitigate its risk and align it to a greater good.

Mission

A body comprised of hundreds of individuals cannot function without a unifying mission. There are many ideas of what this should be for WG21, but here are a few strawmen worth considering:

1. Make/keep C++ the best language in the world.
2. Make C++ the only language people use.
3. Make C++ the most popular language.

The ideas of a “best”, “sole”, or “most popular” language are questionable, but more concerning is their impact. First, this outlook leads to an aversion of other languages both from pride and fear that those other languages might be “better”. Consider, for example, that 40% of C++ developers want to use Rust and 22% already do; ignorance of Rust is ignorance of our users.[2] Second, positioning C++ as the “best” language leads to grafting features of other languages at the expense of complexity and consistency. Finally, a lot of energy is expended in useless arguments claiming the benefits of “competing” languages are overblown and that the drawbacks of C++ are exaggerated.

We need a more constructive mission and I think there is one: to improve people’s lives. When the range-based for loop reached compilers, millions of developers smiled and said “Ah, that’s nice.” It may have even made their day. This is the kind of massive good within WG21’s control and aligning ourselves to it is incredibly rewarding.

An altruistic mindset eliminates the idea of “competitor”. Would Habitat for Humanity morn if the Peace Corps reached a distressed area first? Of course not! They would celebrate the arrival of help. It should be the same with us when our users solve their problem using a different tool. Other language communities helping our users are our allies. We must not forget that. Turf wars don’t serve our users’ interests.

The social aspect

Some patterns of thought frustrate a mission of improving people’s lives. Awareness of these is important as they undoubtedly crop up.

C++ as personal and group identity

One of the first questions programmers ask each other in a social setting is “What language do you program in?”. This frequently sets the stage for unfortunate stereotypes. “Oh, a Java programmer who writes slow code”. “Oh, a Python programmer who cannot program in a ‘real’ language”. “Oh, a Go programmer who …”. When we think of C++ we may find pride in mastery of a difficult language, the ability to write high performance code, and a relationship to the world’s greatest C++ works.

While there are benefits to identification with C++ as a source of greatness and purpose, this comes at a cost. First, being primarily an emotional transference, it clouds reason. At my first committee meeting I was advised to avoid mentioning functional programming lest people dismiss my arguments upon hearing the words. Second, deep identification with C++ can create deep seated fears that C++ will become a “legacy” language making one’s skillset (and person even!) obsolete.

I mention these things because they frequently compromise a standardization mission to improve people’s lives. We need to assess the programming language world without tribalism tempting us to ignore or overcompensate for issues.

Counterproductive rhetoric

C++ is frequently written and talked about as if it were a living thing. Words like “fatal”[3], “fail”[4], “dead”[5], and “death”[6] are common in our literature. When we imagine C++ as a living thing, we naturally associate finite resources (active users), competition (other languages), and death (obsolescence). This thinking is fundamentally inaccurate. C++ is not alive, cannot die, and isn’t competing against anything. It is a merely a tool that is sometimes useful.

We must move away from this thinking. In combination with the transference previously discussed, it generates fear and encourages the idea of enemies. The current lack of cooperation and credit between different language communities is quite unfortunate. At its worst people are put down because of the programming language they associate with.

Let us remember that a) languages don’t battle, people do, b) smearing other languages doesn’t improve people’s lives, and c) C++ living forever is not our goal.

Standardization as personal opportunity vs. stewardship

When first joining the committee, it is easy to see participation as primarily a personal opportunity to gain C++ expertise, rub shoulders with celebrities, and, worse of all, leave a mark on the world by getting a proposal accepted. While all these things do indeed happen, there is something much larger at play here.

The direction group warns “we are writing a standard for millions of programmers to rely on for decades, a bit of humility is in order.”[7] This is not an earnable privilege and none of us are really qualified to make these decisions, but here we are. Our heavy responsibility outweighs the personal opportunities.

What does that responsibility entail? It means rejecting proposals without an understandable value proposition. It means resisting social pressure when you are against something. It means building an informed opinion by reading the paper, testing the feature, and collaborating with others. It means saying “yes” only when there is minimal risk. Above all, it means stewardship: you are a caretaker and guardian of something beyond yourself.

If you do write a proposal, save time and frustration by enlisting the help of experienced designers and wording experts. They want to help! Also, carefully consider if the problem you’re solving justifies the additional complexity and risk. C++ is a language that is “trying to do too much too fast”[8] and needs “to become more restrained and selective”[9].

The technical aspect

Equally important to our social tendencies are the technical tendencies that work against us. This section calls out several anti-patterns, none of which are new [10].

Neophilia

Bjarne succinctly stated “Enthusiasm favors the new”[11]. Technological innovations and fads follow a familiar hype curve [12] that begins with a peak of inflated expectations. We risk getting caught up in enthusiasm and standardizing features that, in retrospect, don’t deliver on their promise, poorly integrate with the rest of the language, and increase learning costs.

Consider Rust traits which solve similar problems to those of C++ concepts. Traits’s explicit opt-in semantics offers several advantages including separately type-checked generics. Should we add traits to C++? If we do so, we’ll end up with two ways to solve the same problem with millions of lines of code using the old way. Moreover, most developers will need to be familiar with both to be effective in a large, existing codebase, compounding C++'s learning costs.

Just because another language has something potentially better than C++ doesn’t mean we should incorporate it. “Keeping up with the Joneses” is a disservice. We should ask ourselves how non-experts, making up most of our users, will react when seeing a feature for the first time in someone else’s code. Frequently it is frustration from having to spend time learning something with marginal benefit over what it replaces.

Features and prioritization bias towards experts

The C++ committee, predominantly comprised of experts, leaves average programmers “seriously underrepresented”[13]. This “biases the committee towards language lawyering, advanced features, and implementation issues, rather than directly addressing the needs of the mass of C++ developers, which many committee members know only indirectly”[14].

Time spent on expert features squanders opportunities to improve lives at scale. When we prioritize a proposal, we should ask “is this solving a problem for experts or for the average developer?”. If it’s the former, we should seriously consider moving on.

Our expert imbalance also results in over-complicated solutions that require advanced proficiencies for simple tasks. Consider the hoops one needs to jump through to make `std::print` work with a custom type when compared to the old stream operators. It is too easy for experts to lose touch with novices and professional engineers who don’t spend their free time learning advanced C++ complexities, especially when surrounded by other experts.

One of the most valuable things committee members can do is discuss proposals with application engineers. “Is this something you would use?”. “Is this ergonomic?”. “How hard is this to learn?”. “Is this worth another chapter in the C++ book?”. This kind of feedback should weigh more heavily than abstract theories on what an ideal developer should want.

Complexity

The direction group sees “C++ in danger of losing coherency due to proposals based on differing and sometimes mutually contradictory design philosophies and differing stylistic tastes.” [15] Bjarne suspects this is due to a combination of committee growth, an influx of new people, specialization of membership, and a decrease of knowledge of C++'s history among the members.[16]

Changes reducing coherence increase complexity and this elevates training costs. Hiring C++ developers is much more challenging than hiring other developers not because of demand, but because the barrier to entry is too high. Fewer people want to learn C++ and fewer schools want to teach it. One of the important ways we can improve people’s lives is to help our users find colleagues.

The direction group recalls Alex Stepanov rescuing C++ from disaster by bringing consistency and coherence to the standard library[17], yet we actively debate breaking these same rules for a relatively niche library addition. We recently replaced a simple `std::function` template with no less than three alternatives: `std::copyable_function`, `std::function_ref`, and `std::move_only_function`. This isn’t helping our complexity problems!

I agree with the design group that we must “aim for coherence”[18]. Here are three concrete suggestions for doing so:

1. Curb the tendency to focus proposal discussions on a narrow problem domain by asking how it fits within the entire C++ offering. “Is this in the common C++ style?”. “Is this increasing C++'s barrier to entry?”. “How would this impact the hypothetical ‘C++ book’?”
2. Encourage study groups to get early feedback from the evolution groups (EWG and LEWG) on the desirability of features.[19] Evolution groups are responsible for considering the bigger picture. Getting this feedback before extensive study group iteration can prevent undesirable features gaining difficult-to-stop momentum.
3. Overcome reluctance to say, “I don’t think this belongs in C++.” We don’t do authors any favors by providing improvement feedback on proposals that are ultimately undesirable.

Niche problems getting more than niche effort

> [I]t is hard for a committee to remember that a language cannot be all things to all people. It is even harder to accept that it cannot solve even the most urgent problems of every member.
>
> Bjarne Stroustrup, *Thriving in a Crowded and Changing World*

In committee, we frequently spend time on things that only a small number of people care about. It’s difficult to say “no” when someone, somewhere would benefit. There’s also a tendency to mentally check-out during these discussions which results in proposals not getting an appropriate rigor.

When we fall into these traps, we 1) deny the greater number of users time spent on proposals that can improve their lives, 2) needlessly increase the complexity of the language and library, and 3) encourage more niche proposals.

Solving these problems boils down to saying “no” more often and, if needed, repeatedly. Bug fixes aside, the committee should spend its time on a fewer number of proposals that have a bigger impact. Effort spent writing papers solving pet peeves in C++ is better spent writing up analyses and experience reports on higher-impact proposals. We should be doing more to acknowledge such work.

Moving ahead

This section considers a memory safety and a major C++ overhaul in light this paper’s principles.

Memory safety

Official documents discussing legislation against C++ due its “memory unsafety” have caused community uproar. We’ve seen gigantic email threads, a new study group dedicated to safety, and numerous talks at C++ conferences. What is much less prevalent is a demand from average C++ users for memory safety features; they’re much more concerned about compilation speed. When most C++ developers haven’t adopted tools like Coverity and C++ core guidelines checkers, it is hard to claim that memory safety features substantially improve their lives at least from their point of view.

Where memory safety *is* a serious concern, we see the adoption of Rust for critical components. Yet we see little demand from even these developers for C++ safety features. Their problem is already solved.

The direction group states “no language can be everything for everybody”[20] and I cannot agree more. Rust and other languages are successfully filling engineering needs for memory safety guarantees in critical components. This is not a space our users are demanding us to go into and doing so risks both failure and, yes, even more complexity.

Major C++ overhaul

Over the past two years it’s become in vogue to talk about “C++ successors” which range from dramatic syntax changes to replacing the C++ committee with another organization. What should the committee’s response be to this phenomenon?

For groups attempting a new language outside of the committee, I think our response should be support. If these initiatives don’t confuse or otherwise harm our users, they share our goal of improving people’s lives. When they succeed, that’s a good thing. Even if they fail, the ideas they generate might help our users in the end.

What about the option to drastically change the face of C++ in the context of WG21? A C++ 2.0, perhaps? If you ask a typical C++ developer how we can improve their lives, a modern and snazzy new syntax will not top their list. Yes, `template` and `typename` are tedious to read and type, but it’s what they know and they’d rather it not be mucked with. This is more than reluctance to change–our users want coherence in their C++ code base as much as we want coherence in the standard.

If a C++ successor ever gains traction, our users would want it to be the best it can be. The committee composition doesn’t have a magical quality that makes it more suited to build a successor than any other entity. The inertia from C++ 1.0’s adoption may even lead to a C++ 2.0 getting adopted when other successor attempts are more fit for purpose. That wouldn’t be good for our users.

In summary, the C++ committee's biggest opportunity to improve people's lives is to focus on C++ as it is today to serve us better in a few years time under the constraints of compatibility. Let's leave speculative successor projects to external entities. We're distracted enough as it is.

What should we do?

I've said a lot about what we should not do. That's intentional–we need to do less. However, I don't want to give the impression we should say no to *all* proposals. There are plenty of opportunities to improve our user's lives through proposals. Here are a few concrete examples:

- **Faster hashing and hash combiners**. The standard library's hash map and hash set interfaces are now decades old. Over that time, we've seen an explosion in their utility and many algorithmic advancements, advancements that require an interface change. By adding more modern hashing data structures to the standard, we can substantially improve the performance, and environmental impact, of newly written code. Our users also desperately need a standardized way to combine hashes in their own hash functions.

- **JSON parsing**. A simple, ergonomic, standardized JSON parsing and serialization library will save many users from searching libraries or, worse, writing custom formats. A non-goal is to be the world's fastest JSON parser.

- **Command-line parsing**. A simple, standardized library for command-line parsing will also improve the lives of the 99% by replacing the common `argv[1]` parsing in small applications.

There are many more ideas like these. The goal is to give as many people as possible the "ah, that's nice" reaction.

Conclusion

This paper advocates for a C++ standardization mission: improving people's lives. It also identified the social and technical biases that obstruct this mission. Finally, it considered major ongoing WG21 discussions and suggested ideas for future work.

In the end, when I say "C++ should be C++" I mean that C++ is a useful tool as it is–drastic changes aren't helpful. To avoid it becoming what it is not, we need to say "no" more often, recognize our biases, and, above all, put our users first.

References

“2023 Developer Survey.” *Stack Overflow*, 2023, https://survey.stackoverflow.co/2023/ (https://survey.stackoverflow.co/2023/).

Hinnant, Howard et al. “Direction for ISO C++.” 15 October 2022, https://wg21.link/P2000R4 (https://wg21.link/P2000R4).

Stroustrup, Bjarne. “How can you be so certain?” 18 November 2019, https://wg21.link/P1962R0 (https://wg21.link/P1962R0).

Stroustrup, Bjarne. “Remember the Vasa!” 6 March 2018, https://wg21.link/P0977R0 (https://wg21.link/P0977R0%5D).

Stroustrup, Bjarne. “Thriving in a Crowded and Changing World: C++ 2006-2020.” *Proc. ACM Program. Lang.*, vol. 4, HOPL, June 2020, Article 70, pp. 1-167, https://doi.org/10.1145/3386320 (https://doi.org/10.1145/3386320).

van Winkel, J.C. et al. “Operating principles for evolving C++” 31 Janurary 2017, https://wg21.link/P0559R0 (https://wg21.link/P0559R0).

1. *Remember the Vasa!* (P0977R0 (https://wg21.link/P0977R0)) ↩

2. The Stack Overflow 2023 survey had 89,184 respondents. 19,634 indicated they did "extensive development work" C++ over the past year and 4,269 claimed extensive development work in both Rust and C++ over the past year. Of those who used C++, 7,918 indicated a desire to work in Rust over the next year. ↩

3. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4) ↩

4. *How can you be so certain* (P1962R0 (https://wg21.link/P1962R0)) ↩

5. *Operating principles for evolving C++* (P0559R0 (https://wg21.link/P0559R0)) ↩

6. ibid. ↩

7. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩

8. *How can you be so certain?* (P1962R0 (https://wg21.link/P1962R0)) ↩

9. ibid. ↩

10. See *Thriving in a Crowded and Changing World* and *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩

11. *Thriving in a Crowded and Changing World* ↩

12. https://en.wikipedia.org/wiki/Gartner_hype_cycle ↩

13. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩

14. *Thriving in a Crowded and Changing World* ↩

15. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩

16. *Thriving in a Crowded and Changing World* ↩

17. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩

18. ibid. ↩

19. Credit to Niall Douglas for this idea ↩

20. *Direction for ISO C++* (P2000R4 (https://wg21.link/P2000R4)) ↩